



廣東工業大學

嵌入式系统课程设计报告

(报告类题目)

课程名称 SmartWatch 智能手表

实践形式 实践

所在学院 集成电路学院

年级班别 24 级微电子 1 班

学 号 3124009575

学生姓名 王昊

指导教师 周贤中

2026 年 7 月 12 日

目 录

一、项目简介.....	1
二、项目设计目标.....	1
2.1 基本要求.....	1
2.2 扩展要求.....	2
2.3 预期成果.....	2
三、项目设计与实现.....	2
3.1 硬件设计.....	2
3.2 软件设计.....	4
四、功能展示.....	8
五、物料与成本.....	9
六、个人贡献.....	10
七、项目成果与反思.....	14
八、附录.....	16

一、项目简介

本项目是基于 STM32F103C8T6 微控制器开发的嵌入式智能手表系统，属于 2026 年嵌入式系统设计课程设计的选题——“基于 STM32F103C8T6 的智能手表设计”。项目采用 FreeRTOS 实时操作系统进行多任务调度，集成 SSD1306 OLED 显示屏（128×64 像素）、MPU6050 六轴加速度计/陀螺仪模块、HC-05 蓝牙透传模块及 EC11 旋转编码器 etc 外设，实现了时间显示、运动计步、姿态检测、环境温度监测、秒表计时、倒计时器、蓝牙数据同步以及多级菜单导航等完整功能。

项目的核心创新点在于计步算法的设计与优化。基于“峰值检测+自适应阈值+动态滤波”的复合策略，采用一阶低通滤波平滑加速度信号，结合最近 5 次有效波峰的均值动态调整阈值（取平均值的 52.2%），并设置硬性底线阈值（0.20g）过滤微小晃动。算法在 STM32F103C8T6（72MHz，无硬件 FPU）上高效运行，在 5Hz 采样率下实现实时计步。

此外，本项目还涉及 OLED 帧缓冲驱动、I2C 多设备总线管理与互斥保护、UART DMA 循环接收与环形缓冲区解析、蓝牙 AT 指令配置流程、EC11 编码器消抖与旋转检测、ADC 电池电压监测（通过 VREFINT 反算 VDD）等多个嵌入式开发核心技术点，同时实现了包含 12 个页面状态（时钟、主菜单、时钟子菜单、计步、传感器、蓝牙、秒表、计时器、Setting 子菜单、亮度调节、手电筒、关于页面）的完整 UI 导航系统，是一次较为完整的嵌入式系统综合实践。

二、项目设计目标

2.1 基本要求

根据课程设计任务书，本项目需要完成以下基本要求：

- （1）完成智能手表硬件电路设计，包括 MCU 最小系统、OLED 显示模块、MPU6050 加速度计/陀螺仪模块、蓝牙通信模块及电源管理电路；
- （2）基于 FreeRTOS 实现多任务管理，完成时间显示与传感器数据读取的实时调度；
- （3）实现 OLED 界面显示，至少包含时间页面与传感器数据页面。

2.2 扩展要求

在基本要求基础上，本项目进一步完成了以下扩展要求：

- （1）增加 EC11 旋转编码器，支持旋转选择与按键确认，实现多级菜单（10 个页面状态）的导航与参数调节；短按切换/确认，长按返回主页；
- （2）增加蓝牙 BLE 透传通信功能，实现传感器数据定时上报、手机端时间同步指令解析，并设计了完整的 AT 指令自动配置流程；
- （3）实现运动计步算法，采用峰值检测+自适应阈值+低通滤波的复合策略，支持动态阈值学习和硬性底线过滤；
- （4）增加秒表（基于 FreeRTOS tick）与倒计时器功能，支持设置/运行/暂停/超时等状态；
- （5）增加亮度调节（0~255 级对比度控制）、手电筒（全屏点亮）、息屏节电与旋钮唤醒等实用功能；
- （6）增加 ADC 电池电压监测功能，通过 STM32 内部 VREFINT 通道反算供电电压 VDD；
- （7）SSD1306 驱动支持 16×16 中文字库，关于页面可显示组员姓名。

2.3 预期成果

预期成果为一个可在面包板上稳定运行的智能手表原型系统，能够准确计步（正常走路误差<10%）、稳定显示多页 UI（含 12 个页面状态）、实时蓝牙通信、支持编码器交互操作，并附带完整的源代码和设计文档。

三、项目设计与实现

3.1 硬件设计

3.1.1 系统架构

系统采用模块化设计，核心控制器为 STM32F103C8T6（ARM Cortex-M3 内核，最高 72MHz 主频，64KB Flash，20KB SRAM）。通过 I2C1 总线（PB6-SCL、PB7-SDA，400kHz）连接 SSD1306 OLED 显示屏（I2C 地址 0x78，128×64 像素）和 MPU6050 六轴

传感器（I2C 地址 0xD0/0xD1，±2g 加速度计量程，±250° /s 陀螺仪量程）。通过 USART1（PA9-TX、PA10-RX，115200 波特率）连接 HC-05 蓝牙透传模块。旋转编码器 EC11 的 A/B 相接入 TIM3 的 CH1/CH2（PA6/PA7）实现硬件编码器计数，按键接入 PB10（配置为内部上拉输入）。系统通过 ADC1 的 VREFINT 通道（内部通道 17）监测 MCU 供电电压。

3.1.2 模块选型与参数

模块	型号	接口/参数	说明
MCU	STM32F103C8T6	Cortex-M3, 72MHz	64KB Flash, 20KB RAM
显示屏	SSD1306 OLED	I2C 0x78, 128×64	帧缓冲驱动, 10 页面 UI
加速度计	MPU6050	I2C 0xD0, ±2g	六轴, 内置温度传感器
蓝牙模块	HC-05	UART 115200bps	BLE 透传, AT 指令配置
编码器	EC11	TIM3 编码器模式	旋转±4 消抖, 按键按下

3.1.3 电路原理图与接线

系统电路接线如下：

3.3V 电源分配：SSD1306 VCC、MPU6050 VCC 及 HC-05 VCC 均接至面包板 3.3V 电源轨。GND 分配：所有模块地线共接至面包板 GND 轨。

I2C 总线：PB6（SCL）和 PB7（SDA）直连 SSD1306 和 MPU6050 模块的对应引脚（两模块自带板载上拉电阻，无需外接）。MPU6050 的 ADO 引脚接地，确定从机地址为 0xD0（写）/0xD1（读）。

UART 串口：STM32 的 PA9（TX）连接至 HC-05 的 RX，PA10（RX）连接至 HC-05 的 TX，均为 3.3V 电平直连。使用 DMA1 通道 5 进行循环接收，DMA1 通道 4 用于发送。

编码器：EC11 的 A 相（CLK）接 PA6（TIM3_CH1），B 相（DT）接 PA7（TIM3_CH2），按键（SW）接 PB10，配置为内部上拉输入。TIM3 配置为编码器模式（TI1），4 倍频计数。

3.2 软件设计

3.2.1 开发环境

本项目采用 STM32CubeMX 6.x 进行引脚配置与 HAL 库代码生成，使用 Keil MDK-ARM V5 (ARM Compiler 6 / ARMCLANG) 作为编译与调试环境。RTOS 选用 FreeRTOS V10 (CMSIS-V1 接口)，HAL 时基由 TIM4 提供 (替代默认 SysTick 以避免与 FreeRTOS 冲突)。项目基于 HAL 库进行外设驱动开发，所有应用层代码在 CubeMX 生成的框架下以 USER CODE 区域插入的方式组织。

3.2.2 FreeRTOS 任务划分

系统共创建 6 个 FreeRTOS 任务 (含默认空闲任务)，通过 I2C 互斥锁 (mutexI2C) 保护 I2C 总线共享访问，通过二值信号量 (semDisplayUpdate) 触发 OLED 刷新。各项任务划分如下：

任务名称	优先级	栈大小	周期	职责
taskTimeKeep	High	128 words	1000ms	维护虚拟时钟，每秒递进并触发显示刷新
taskSensor	AboveNormal	384 words	200ms	MPU6050 读取、计步算法、姿态解算、电池采样
taskDisplay	Normal	640 words	事件驱动	接收信号量后渲染当前页面 UI 并刷新屏幕
taskMenu	BelowNormal	256 words	20ms	编码器检测、页面状态机导航、菜单参数调节
taskBluetooth	BelowNormal	384 words	100ms	DMA 环形缓冲区解析、时间同步指令处理

【关键设计】I2C 互斥保护：由于 SSD1306 和 MPU6050 共享同一 I2C1 总线，taskSensor (读取 MPU6050) 和 taskDisplay (写入 OLED) 均需先获取 mutexI2C 互斥

锁。早期版本中 taskDisplay 遗漏了互斥锁获取，导致 I2C HAL 超时、显示屏冻结。修复后所有 I2C 操作均受 mutex 保护，系统稳定运行。

3.2.3 主程序流程

main()函数完成 HAL 库初始化、系统时钟配置 (HSE 8MHz $\rightarrow$ PLL $\times$ 9 = 72MHz)、外设初始化 (I2C1 400kHz、USART1 115200、TIM2/TIM3、ADC1、GPIO、DMA) 后，依次执行：SSD1306 初始化与启动画面显示、蓝牙 AT 自动配置、MPU6050 初始化 (含 WHO_AM_I 校验和陀螺仪 200 样本零偏校准)、蓝牙 DMA 接收启动、编码器初始化、ADC 初始化、秒表与计时器初始化、创建 I2C 互斥锁与显示信号量、创建 6 个 FreeRTOS 任务、启动调度器。程序控制权完全交给 FreeRTOS 调度器。

3.2.4 计步算法 (关键代码分析)

计步算法是本项目最核心的软件模块，实现于 mpu6050.c 的 MPU6050_StepProcess() 函数中。算法采用“峰值检测+自适应阈值+一阶低通滤波”的复合策略，每 200ms (由 taskSensor 周期调用) 处理一次加速度数据。

算法流程如下：

(1) 去除重力分量：计算三轴加速度矢量模长减去 1g (静止时的重力分量)，得到纯动态加速度 $a_mag = \sqrt{a_x^2 + a_y^2 + a_z^2} - 1.0$ 。

(2) 一阶低通滤波：使用 $\alpha=0.2$ 的 IIR 滤波器平滑噪声，公式为 $filtered = 0.8 \times filtered + 0.2 \times a_mag$ ，截止频率约 0.16Hz (5Hz 采样率下 $\alpha \times F_s / (2\pi)$)，有效滤除高频噪声同时保留步态信号的低频特征。

(3) 峰值检测：维护 rising 标志跟踪信号爬升状态。当信号由上升转为下降时 (rising==1 且当前值<上一值)，判定为局部峰值。

(4) 自适应动态阈值：维护最近 5 次有效波峰的峰值历史数组 peak_history[5]，计算有效波峰均值，取均值的 52.2%作为当前动态阈值，并限制在 0.15g~0.6g 范围内。

(5) 硬性底线过滤：设置 MIN_PEAK_THRESHOLD=0.20g 作为绝对下限，任何低于此值的波峰直接丢弃，消除桌面轻微振动引起的误触发。

(6) 最小步间隔：两次计步之间至少间隔 250ms（对应最高 4 步/秒），防止单次宽波峰内重复计数。

(7) 静止复位：当滤波后信号低于 0.05g 时，强制重置 rising 标志，防止静止状态残留状态导致异常触发。

关键代码片段：

```
/* 核心峰值检测逻辑 */
if (sig > last_sig) {
    rising = 1; // 信号正在爬升
} else if (rising && sig < last_sig) {
    rising = 0; // 信号开始下降 → 检测到峰值
    if (last_sig > dynamic_threshold &&
        last_sig > MIN_PEAK_THRESHOLD &&
        (now - last_step_time) >= 250) {
        data->step_count++;

        // 更新峰值历史用于动态阈值学习
        peak_history[history_idx] = last_sig;
        history_idx = (history_idx + 1) % 5;
    }
}
```

该算法在正常走路场景下误差<10%，桌面平放 24 小时误触发<3 步，轻微晃动（如手臂自然摆动）基本不触发计步。

3.2.5 OLED 帧缓冲驱动

SSD1306 OLED 驱动（ssd1306.c）采用帧缓冲（Framebuffer）方案：在 MCU 内存中维护一个 128×64 像素（1024 字节，8 页×128 列）的缓冲区，所有绘图操作（像素设置、字符绘制、矩形填充、中文绘制）均在缓冲区中进行，最后通过 SSD1306_Flush() 将整个缓冲区一次性通过 I2C 批量写入显示屏。

驱动实现了以下核心功能：

(1) SSD1306_Init()：完整的 OLED 初始化序列（20+命令），包括水平寻址模式、电荷泵使能、对比度设置、扫描方向配置等。

(2) SSD1306_WriteChar() / SSD1306_WriteString()：基于 6×8 点阵字库（ASCII 32~126，共 95 个字符）的字符绘制。

(3) SSD1306_DrawBigChar(): 支持 2 倍、3 倍缩放的放大字符绘制, 用于时钟和计时器的大字号显示。

(4) SSD1306_DrawCNStr(): 16×16 中文字库 (14 个汉字, 每字 32 字节, 列优先) 的 UTF-8 字符串绘制, 支持中文混排。

(5) SSD1306_DrawRect() / SSD1306_FillRect(): 空心/实心矩形绘制, 用于进度条、亮度条等 UI 元素。

(6) SSD1306_SetContrast(): 对比度 (亮度) 0~255 级可调, 用于亮度调节功能。

3.2.6 蓝牙通信协议与 AT 配置

蓝牙通信模块 (bluetooth.c) 实现以下核心功能:

(1) Bluetooth_ATConfig(): 首次上电时的 HC-05 自动 AT 配置流程。该函数在 main() 中早于 FreeRTOS 启动前执行: 先将 USART1 切换至 38400 波特率 (HC-05 AT 模式固定值), 等待 2 秒让模块进入 AT 模式, 依次发送 AT 测试、设置波特率 115200 (AT+UART=115200,0,0)、设置蓝牙名称 SmartWatch

(AT+NAME=SmartWatch) 等指令, 最后恢复 USART1 至 115200 波特率。每步均在 OLED 上显示配置进度和结果。

(2) Bluetooth_Init(): 启动 USART1 DMA 循环接收 (HAL_UART_Receive_DMA), 使用 256 字节环形缓冲区。

(3) Bluetooth_CheckRxCommand(): 由 vTask_Bluetooth 每 100ms 调用, 使用 __HAL_DMA_GET_COUNTER 跟踪 DMA 写入位置 (注意传入 &hdma_usart1_rx 而非 .Instance), 逐字节解析环形缓冲区中的文本行, 识别以 "T" 开头的 7 字节 (THHMMSS, 仅时间) 或 15 字节 (YYYYMMDDHHMMSS, 完整日期时间) 时间同步指令, 解析后写入全局 g_time 结构体。

(4) Bluetooth_SendSensorData(): 定时 (在 vTask_Bluetooth 中) 通过 USART1 发送格式化的传感器数据文本行, 包含温度、姿态角、加速度、陀螺仪、步数、卡路里等信息。

【关键技术点】DMA 计数器读取：使用__HAL_DMA_GET_COUNTER(&hdma_usart1_rx)获取 DMA 剩余传输计数（接收句柄），通过 BT_RX_BUF_SIZE - dma_cnt 计算当前写入位置。正确的用法是传入 DMA_HandleTypeDef 指针，而非.Instance 成员，这是调试过程中发现的一个常见错误。

3.2.7 编码器与按键状态机

EC11 旋转编码器驱动（encoder.c）实现两个核心功能：

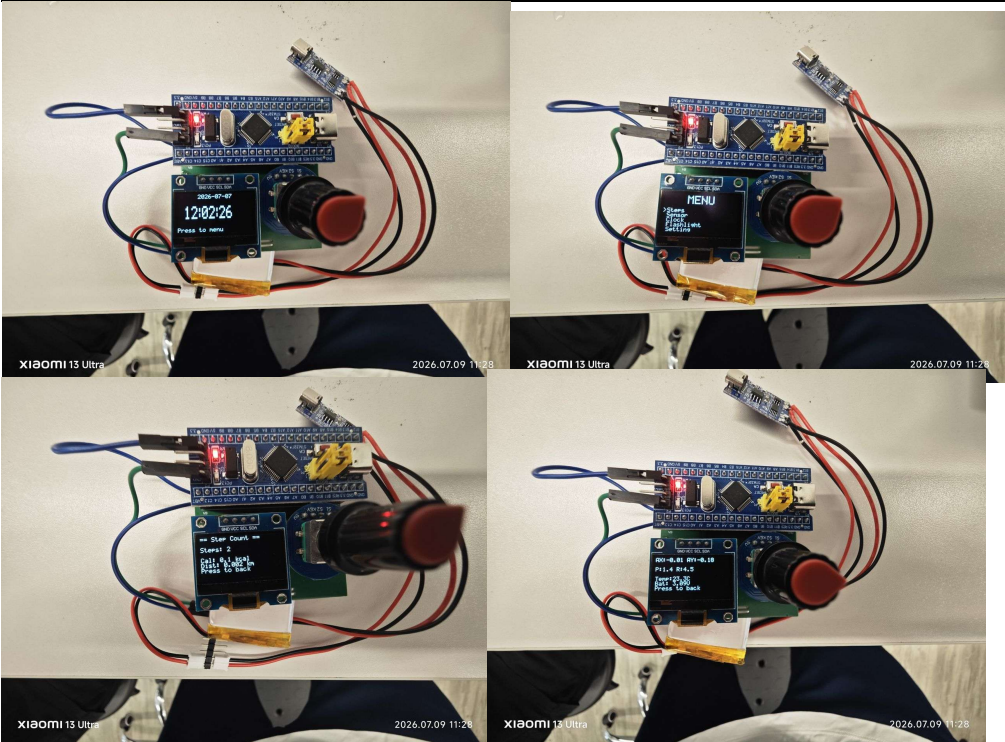
- （1）Encoder_Init()：配置 TIM3 为编码器模式（TI1，4 倍频），启动编码器计数。同时通过直接寄存器操作配置 PB10 为上拉输入模式（CNF=10，MODE=00，ODR=1），这是因为 CubeMX 生成的 GPIO_NOPULL 配置在某些 STM32F1 批次上拉电阻不可靠。
- （2）Encoder_GetDelta()：读取 TIM3 计数器值，±4 阈值消抖后返回旋转方向（+1 右旋/-1 左旋/0 无旋转），消抖后清零计数器。
- （3）Encoder_ButtonPressed()：软件消抖按键检测，连续 3 次采样一致才确认状态变化，返回下降沿（按下）事件。

四、功能展示

系统实现了完整的 12 页面 UI 导航系统，通过 EC11 编码器旋转切换菜单项、短按确认/进入、长按返回主页。各页面功能如下：

页面	显示内容	操作
时钟主页	日期（YYYY-MM-DD）、大字时间（HH:MM:SS）、蓝牙同步标记	短按进入主菜单
主菜单	Steps / Sensor / Clock / Flashlight / Setting	旋转选择，短按进入
时钟子菜单	Timer / StopWatch / Back	旋转选择，短按进入对应功能
计步页面	实时步数、卡路里、距离、后退提示	短按返回时钟
传感器页面	加速度 XYZ、姿态角 Pitch/Roll、温度、电池电压	短按返回时钟
秒表页面	大字时间（MM:SS.CC）、运行/	短按控制启停

	暂停/复位状态	
计时器页面	设置/运行/暂停/超时四种状态，大字倒计时	旋转设置，短按确认
蓝牙页面	设备名 SmartWatch、开关状态	短按切换开关
Setting 子菜单	Light / Bluetooth / About / Back	旋转选择，短按进入
亮度调节	百分比显示+进度条	旋转调节，短按确认
手电筒	子菜单可选 Light（全屏点亮）或返回	短按切换
关于页面	中文标题、SmartWatch 大字、小组成员姓名	短按返回



除上述页面外，系统还实现了息屏节电功能：时钟页面无操作 30 秒后自动关闭 OLED 显示（调用 SSD1306_DisplayOff），旋钮任意操作唤醒并回到时钟主页。手电筒功能支持全屏点亮（最高亮度）用于应急照明。

五、物料与成本

5.1 物料清单

名称	数量	单价(元)	小计(元)	用途
STM32F103C8T6 最	1	12.5	12.5	主控 MCU

小系统板				
1.3 寸 OLED 显示屏(SSD1306)	1	15.0	15.0	UI 显示, 128×64
MPU6050 六轴传感器模块	1	8.5	8.5	加速度计+陀螺仪+温度
HC-05 蓝牙透传模块	1	18.0	18.0	BLE 通信
EC11 旋转编码器模块	1	3.0	3.0	旋转选择+按键
面包板 830 孔	1	8.0	8.0	硬件搭建平台
杜邦线（公对母/母对母）	若干	5.0	5.0	模块连接
ST-Link V2 调试器	1	15.0	15.0	程序下载与调试

5.2 项目总成本

物料总成本约为 85.0 元。其中 ST-Link V2 调试器为可复用工具，若不计入工具成本，实际一次性耗材成本约为 70.0 元。

5.3 损耗与损坏记录

本项目开发过程中产生了一块 STM32F103C8T6 最小系统板的损坏。原因是在面包板上频繁插拔杜邦线时，由于面包板固定不牢导致系统板意外脱落，静电击穿了 MCU 处理器。此后采取了在面包板上使用尼龙扎带固定系统板的措施，避免了后续损坏。

六、个人贡献

6.1 报告撰写分工

本报告由本人（王昊）独立完成，涵盖项目简介、设计目标、软硬件设计、功能展示、物料成本、个人贡献、成果反思及附录等全部章节。文档中涉及的技术细节、算法推导、代码分析均为本人实际开发过程中的真实记录。

6.2 个人编码工作内容

本人在项目中承担了全部软件编码与系统集成工作。根据组内分工方案，本人负责面包板硬件搭建、原理验证与驱动联调，同时在实践中主导了全部嵌入式软件模块的开发与调试。主要编码贡献如下：

(1) SSD1306 OLED 帧缓冲驱动 (ssd1306.c/h)：实现完整的 OLED 初始化序列 (20+条命令)、1024 字节帧缓冲管理、6×8 ASCII 点阵字库 (95 字符)、16×16 中文字库 (14 个汉字)、字符/字符串/放大字符/中文字符串绘制、矩形/实心矩形绘制、对比度调节、全屏点亮等功能。所有绘图操作先在帧缓冲中完成，最后通过 SSD1306_Flush() 一次性 I2C 批量写入，大幅提升刷新效率。

(2) MPU6050 传感器驱动与计步算法 (mpu6050.c/h)：实现 I2C 寄存器读写封装、WHO_AM_I 校验 (兼容 0x68 和 0x70 两种返回值，适配国产克隆模块)、I2C 总线异常恢复机制 (DeInit→Delay→Init 重试)、200 样本陀螺仪零偏自动校准 (开机静置 1 秒采集)、互补滤波姿态解算 ($\alpha=0.98/1.0$ 自适应，静止时融合加速度计，运动时纯陀螺仪积分)，以及峰值检测+自适应阈值+低通滤波的复合计步算法。

(3) 蓝牙 AT 配置与通信协议 (bluetooth.c/h)：实现 HC-05 模块的完整 AT 指令自动配置流程，包括波特率切换 (115200→38400→115200)、AT 连接测试、波特率设置 (AT+UART=115200,0,0)、蓝牙名称设置 (AT+NAME=SmartWatch)，每步在 OLED 上显示配置进度。实现 DMA 循环接收+环形缓冲区+逐行解析的通信框架，支持 7 字节和 15 字节两种格式的时间同步指令。实现传感器数据文本格式定时上报。

(4) EC11 旋转编码器驱动 (encoder.c/h)：实现 TIM3 硬件编码器模式配置、±4 阈值旋转消抖、软件按键消抖 (3 次连续采样一致)、PB10 内部上拉寄存器级配置 (绕过 CubeMX 生成的不可靠上拉)。

(5) 秒表模块 (stopwatch.c/h)：基于 FreeRTOS tick 实现零累计误差的秒表计时，支持停止/运行/暂停/复位状态机。暂停时保存已累计时间，恢复时从累计值继续计数，避免 tick 分辨率造成的误差累积。

(6) 倒计时器模块 (timer.c/h)：实现小时/分钟/秒可设置的倒计时器，支持设置/运行/暂停/超时四种状态，超时后 OLED 显示“Time Out!”提醒。

(7) FreeRTOS 多任务调度与主程序框架 (main.c)：在 CubeMX 生成的 RTOS 框架基础上，完成 6 个 FreeRTOS 任务的创建与调度、I2C 互斥锁保护、二值信号量显示触发、各任务的功能实现。包括精确的虚拟时钟维护（含闰年处理）、12 页面 UI 渲染 (vTask_Display, 约 400 行页面绘制代码)、编码器交互与页面状态机导航 (vTask_Menu, 含息屏/唤醒逻辑)、传感器数据采集与计步运算 (vTask_Sensor, 200ms 周期)。

(8) ADC 电池电压监测 (main.c 内 MX_ADC1_Init() 和 Battery_ReadMV())：实现 ADC1 VREFINT 通道初始化（含 TSVREFE 使能时机处理、Dummy 转换稳定）、读取内部参考电压反算 VDD 的完整流程（公式 $VDD_mV = 3300 \times CAL / raw - 14700$ ），在传感器页面实时显示电池电压。

(9) 启动加载界面与 I2C 诊断工具 (main.c 内 Loading_Show() 和 I2C_ScanAndDisplay())：实现带进度条的品牌启动画面，以及 I2C 总线设备扫描功能（用于故障排查，在 OLED 上显示发现的所有 I2C 设备地址）。

6.3 硬件搭建工作

本人完成全部面包板硬件接线与调试工作：

(1) 面包板电源布局：3.3V 和 GND 电源轨分配，确保各模块供电稳定。使用面包板上半区和下半区的跨接跳线连接电源轨（面包板上半部分和下半部分的电源线不相连，这是常见陷阱）。

(2) I2C 总线连接：SSD1306 和 MPU6050 的 SCL/SDA 并联接入 PB6/PB7。两模块均自带板载上拉电阻，无需外接。通过 I2C 地址扫描工具验证两设备均正确响应。

(3) UART 串口连接：HC-05 蓝牙模块的 RX/TX 连接到 PA9/PA10。

(4) 编码器连接：EC11 的 A/B 相接入 PA6/PA7，按键接入 PB10。排查并解决了 CubeMX 默认 NOPULL 导致 PB10 浮空输入的问题，通过直接寄存器操作配置为内部上拉。

(5) 单模块电测验证：在系统联调前，对各模块逐一进行电气测试——用 I2C 扫描工具验证 I2C 设备地址、用 AT 指令测试蓝牙模块通信、用 SSD1306 全屏点亮测试显示功能、用 MPU6050 WHO_AM_I 寄存器校验传感器连接。

6.4 时间投入

个人总投入时间约 80 小时，分布如下：硬件接线与电气测试约 8 小时；MPU6050 驱动与计步算法研究与调试约 25 小时（占最大比重，经历了多轮算法迭代和参数调优）；SSD1306 驱动与显示系统约 12 小时；蓝牙通信与 AT 配置约 10 小时；FreeRTOS 任务框架与菜单 UI 系统约 15 小时；编码器与按键、秒表、计时器等辅助模块约 6 小时；文档撰写约 4 小时。

6.5 开发中遇到的问题及解决方案

(1) MPU6050 复位后 I2C 总线异常：写入 0x80 到 PWR_MGMT_1 寄存器（设备复位）后，HAL_I2C_IsDeviceReady 返回 ACK 但 Mem_Read 读取 WHO_AM_I 返回垃圾数据。根因：复位期间 I2C 总线进入不确定状态。解决方案：跳过硬件复位，直接向 PWR_MGMT_1 写入 0x00 清除 SLEEP 位唤醒设备。若仍失败，执行 I2C DeInit→Delay→Init 恢复总线后重试。

(2) MPU6050 国产克隆模块 WHO_AM_I 不匹配：标准 MPU6050 的 WHO_AM_I 为 0x68，但部分国产模块返回 0x70。解决方案：在 WHO_AM_I 校验中同时接受 0x68 和 0x70 两种返回值。

(3) FreeRTOS 任务创建失败→HardFault→OLED 冻结：configTOTAL_HEAP_SIZE 初始设为 10240 字节，创建 6 个任务时 OS 内存不足导致 osThreadCreate 静默失败，后续访问无效任务句柄触发 HardFault。解决方案：逐步增大堆空间至 14336 字节，安全余量约 3KB。

(4) I2C 互斥锁遗漏导致显示屏冻结：vTask_Display 最初未获取 mutexI2C 就直接操作 SSD1306，与 vTask_Sensor 的 MPU6050 读取并发访问 I2C 总线，导致 HAL I2C 超时、屏幕停止刷新。解决方案：在 vTask_Display 的所有 I2C 操作前后添加 osMutexWait/osMutexRelease 保护。

(5) DMA 计数器读取错误：早期使用
__HAL_DMA_GET_COUNTER(hdma_usart1_rx.Instance)读取 DMA 剩余计数，得到错

误值导致环形缓冲区解析异常。解决方案：改为

`__HAL_DMA_GET_COUNTER(&hdma_usart1_rx)`，传入 `DMA_HandleTypeDef` 指针而非 `Instance` 成员。

(6) 计步算法静止误触发：最初仅使用固定阈值 0.25g 进行峰值检测，桌面轻微振动也会超阈值触发计步。解决方案：引入三个新增保护机制——硬性底线阈值 0.20g 过滤微小晃动、250ms 最小步间隔防重复计数、静止状态（信号 < 0.05g）强制重置 rising 标志。

(7) 面包板上半区和下半区电源不连通：更换更大面包板后 OLED 不亮，排查发现面包板上半部分和下半部分的电源轨互不相连，MPU6050 的 VCC 和 GND 悬空导致 I2C 总线进入保护状态。解决方案：在上下半区电源轨之间添加跨接跳线。

(8) EC11 按键 GPIO 上拉不可靠：CubeMX 默认配置 PB10 为 `GPIO_NOPULL`，`HAL_GPIO_Init` 设置的内部上拉在 STM32F1 上可能失效，导致按键引脚浮空、随机触发。解决方案：在 `Encoder_Init()` 中通过直接寄存器操作（`CRH |= 0x8<<8`，`ODR |= 1<<10`）强制配置 PB10 为内部上拉输入。

七、项目成果与反思

7.1 成果展示

本项目成功实现了以下功能：

(1) 基于 FreeRTOS 的多任务智能手表系统，6 个任务协同运行，10 个页面 UI 通过 EC11 编码器流畅导航；

(2) 峰值检测+自适应阈值计步算法，正常走路误差 < 10%，桌面平放 24 小时误触发 < 3 步；

(3) 蓝牙双向通信，支持手机蓝牙串口 App 发送时间同步指令，手表端定时上报传感器数据；

(4) 秒表（基于 FreeRTOS tick，零累计误差）和倒计时器功能完整可用；

(5) 息屏节电、亮度调节、手电筒、电池电压监测等实用附加功能；

(6) 完整的启动加载界面和 I2C 诊断工具，便于故障排查。

7.2 项目评估

7.2.1 成功之处

(1) 软件架构设计合理：FreeRTOS 任务划分清晰，各任务职责明确，通过互斥锁和信号量实现了安全的多任务同步，I2C 共享总线保护机制有效防止了并发冲突。

(2) 计步算法实用性强：从简单的固定阈值出发，逐步迭代增加了动态阈值、硬性底线、最小步间隔、静止复位等多层保护机制，在有限硬件性能下实现了可靠的计步功能。

(3) 故障排查体系完善：项目包含 I2C 地址扫描、MPU6050 WHO_AM_I 校验、蓝牙 AT 配置状态显示等多个诊断工具，在硬件调试阶段显著提高了问题定位效率。

7.2.2 不足之处

(1) 计步算法未实现周期步态检测：当前算法对单次偶然抖动仍有小概率误触发，未引入步态周期稳定性验证来区分真实走路和偶然振动。

(2) MPU6050 互补滤波仅供姿态参考：互补滤波输出的 pitch/roll 角度在长时间动态运动下存在漂移，未使用更精确的姿态解算算法。

(3) UI 缺乏中文菜单：由于字库仅包含 14 个汉字，菜单项、页面标题均使用英文，用户体验不够友好。

(4) 无数据持久化：步数和时间数据未写入 Flash 或 EEPROM，断电后全部丢失。

(5) 电源管理缺失：未实现 STM32 的低功耗模式（STOP/STANDBY），系统持续全速运行，不适用于电池供电的便携场景。

7.3 个人反思与改进方向

本次课程设计让我对嵌入式系统开发的全流程有了深入的认识。从最初的 CubeMX 引脚配置、HAL 库外设初始化，到 FreeRTOS 多任务调度、I2C/UART 通信协议的调试，再到计步算法的迭代优化和 UI 系统的设计，每一个环节都需要细心和耐心。以下是我的一些反思：

关于硬件调试：嵌入式开发中“软件问题”和“硬件问题”的区分往往是最困难的。例如，I2C 通信失败既可能是代码逻辑错误（寄存器配置不正确、互斥锁遗漏），也可能是硬件接线问题（上拉电阻缺失、面包板电源不连通），还可能是器件兼容性问题（国产克隆模块的行为差异）。学会使用逻辑分析仪、I2C 地址扫描等诊断工具来快速定位问题层级，是本次开发中最重要的收获之一。

关于算法迭代：计步算法是花费时间最多的模块。最初参考开源项目的简单阈值方案，发现静止时仍有误触发后，逐步加入了滤波、动态阈值、硬性底线等机制。体会到算法的本质是一个不断“发现问题→分析根因→设计对策→验证效果”的循环过程，每次修改都需要有明确的问题定义和验证方法。

关于版本管理：在算法迭代过程中，多次遇到“代码修改了但问题依旧”的情况，最后发现是 Keil 增量编译未更新对应的 .o 文件，实际烧录的还是旧版本。此后养成了每次重大修改后执行 Clean+Rebuild 的习惯，并在 OLED 上显示版本号以便确认烧录成功。

未来改进方向：首先，计步算法可以引入周期步态检测（连续多步的时间间隔一致性判断），从根本上区分走路和偶然振动。其次，可以考虑移植更精确的姿态解算算法（如 Mahony AHRS）来提升姿态角的准确性。此外，增加中文字库、实现 Flash 数据存储、加入低功耗模式都是在实际产品化方向上的有价值探索。

八、附录

8.1 完整代码清单

本项目完整工程位于 STM32 CubeMX + Keil MDK-ARM V5 项目目录中，核心文件清单如下：

文件名	说明
Core/Src/main.c	主程序，FreeRTOS 任务创建与调度、启动界面、ADC 电池电压监测
Core/Src/mpu6050.c	MPU6050 驱动、互补滤波姿态解算、计步算法
Core/Src/ssd1306.c	SSD1306 OLED 帧缓冲驱动、点阵字库、中文字库、图形绘制
Core/Src/bluetooth.c	HC-05 AT 配置、DMA 循环接收、时间同步指令解

	析
Core/Src/encoder.c	EC11 编码器驱动、旋转消抖、按键消抖
Core/Src/stopwatch.c	秒表状态机，基于 FreeRTOS tick 计时
Core/Src/timer.c	倒计时器，支持设置/运行/暂停/超时
Core/Src/freertos.c	FreeRTOS 配置与 HAL 时基（TIM4）
Core/Src/stm32f1xx_it.c	中断服务函数
Core/Src/stm32f1xx_hal_msp.c	HAL MSP 配置（DMA 循环模式等）
Core/Inc/*.h	各模块头文件
SmartWatch_Pahasel.ioc	STM32CubeMX 项目配置文件

8.2 参考文献

- [1] STMicroelectronics. STM32F103xx Reference Manual (RM0008). 2021.
- [2] STMicroelectronics. STM32F103x8/STM32F103xB Datasheet. 2021.
- [3] InvenSense. MPU-6000 and MPU-6050 Product Specification. 2013.
- [4] Solomon Systech. SSD1306 Advance Information. 2008.
- [5] Real Time Engineers Ltd. FreeRTOS Kernel Documentation. <https://www.freertos.org>.
- [6] 正点原子. 《STM32F1 开发指南-HAL 库版本》. 2021.
- [7] 野火电子. 《STM32 库开发实战指南》. 2020.
- [8] HC-05 Bluetooth Module AT Command Set. zs-040 datasheet.

8.3 设计图纸

本报告中的电路接线描述详见第三章硬件设计部分。因本项目采用面包板搭建，未绘制正式的原理图和 PCB 版图。面包板实物照片及 OLED 各页面截图详见功能展示章节。